

ÇÖZÜM TASARIM DÖKÜMANI
(SOLUTION DESIGN DOCUMENT – SDD)

PROJE ADI

YAPAY ZEKA DESTEKLİ TASARIM
OPTİMİZASYON VE SEO SÜRECİ

HAZIRLAYAN

OĞUZHAN EVCİ

TARİH

15 Eylül 2026

İçindekiler

İçindekiler.....	1
1. Mimari ve Çalışma Ortamı.....	2
1.1 Çalışma Modeli ve Mimari.....	3
1.2 Robot Tipi ve Tetikleme Mekanizması	3
1.3 Kullanılan Teknolojiler.....	3
1.4 Orchestrator Ortamı	3
2. Uygulama ve Sistem Arayüzleri	3
2.1 Dosya ve Dizin Yapısı (OS Workspace)	3
2.2 Python Motoru.....	4
2.3 Gemini Yapay Zeka Servisi.....	4
2.4 Microsoft Excel	4
2.5 GSuite E-Posta Servisi.....	4
3. Teknik Süreç Akışı (Technical Process Flow).....	5
3.1 Dispatcher Akışı	5
3.2 Performer Akışı	5
3.2.1 Init Hazırlık Aşaması (Initialization Phase).....	5
3.2.1.1 Ayarların ve Varlıkların Okunması (InitAllSettings).....	5
3.2.1.2 Açık Uygulamaların Kapatılması (KillAllProcesses)	6
3.2.1.3 Klasör ve Sistem Doğrulaması (InitAllApplications)	6
3.2.1.4 Günlük Excel Takip Tablosunun Hazırlanması	7
3.2.1.5 Python Motoru Varlık Kontrolü	7
3.2.2 Veri Alma Aşaması (Get Transaction Data)	8
3.2.2.1 Kuyruktan Gelen Dosyanın Fiziksel Doğrulanması ve Taşınması	8
3.2.2.2 Akıllı Kimlik, Raporlama Alanları ve Global Etiketleme (Transaction Field Assignments).....	9
3.2.3 İşlem Aşaması (Process)	9
3.2.3.1 Format Kontrolü (Business Exception Check).....	10
3.2.3.2 Görsel Kalitesinin Artırılması (Python_Calistir)	10
3.2.3.3 Gemini ile SEO İsimlendirme (Gemini_Baglan)	12
3.2.3.4 Yeniden Adlandırma ve Taşıma (Seq - Yeniden Adlandır ve Taşı)	13
3.2.4 Hatalı Dosyanın İstisna Klasörüne Taşınması (SetTransactionStatus).....	14
3.2.4.1 İş Kuralı Hatası Kurulumu (Business Exception).....	14

3.2.4.2 Sistem Hatası Kurulumu (System Exception).....	15
3.2.4.3 Yeni Dosya Adını Kuyruğa Ekleme (Success)	15
3.2.5 Sürecin Sonlandırılması ve Kapanış (End Process)	16
3.2.5.1 Veri Çekme ve Tablo Oluşturma (Sequence)	17
3.2.5.2 Excel Raporlama (Sequence)	17
3.2.5.3 Gemini Genel Özet Analizi (Sequence)	18
3.2.5.4 Mail Bildirimi ve Kapanış (Sequence).....	18
3.2.5.5 Uygulamaların Kapatılması (CloseAllApplications)	18
4. Veri Modeli ve Konfigürasyon	19
4.1 Yapılandırma ve Assets Yönetimi.....	19
4.2 Kuyruk Mimarisi ve Veri İletişimi	21
5. Hata Yönetimi (Exception Handling)	22
5.1 System Exception	22
5.2 Business Exception.....	22
6. Güvenlik ve Varlıklar	23
6.1 Varlık Yönetimi	23
6.2 Sistem Bağlantıları	23
6.3 Hata ve İstisna Güvenliği	23

1. Mimari ve Çalışma Ortamı

1.1 Çalışma Modeli ve Mimari

Süreç, UiPath REFramework altyapısı üzerine inşa edilmiştir. Sistem, verileri toplayan Dispatcher ve bu verileri kuyruktan alıp işleyen Performer olmak üzere iki ayrı proje olarak tasarlanmıştır.

1.2 Robot Tipi ve Tetikleme Mekanizması

Sistemde kullanılan robot tipi **Unattended Background Process** olarak belirlenmiştir. Dispatcher projesi Orchestrator üzerinde zamanlayıcı ile veya ihtiyaç durumunda manuel olarak tetiklenerek çalıştırılabilir. Performer projesi ise kuyruğa yeni bir veri eklendiğinde otomatik olarak işlemlere başlar.

1.3 Kullanılan Teknolojiler

UiPath Studio, UiPath Orchestrator, Python 3.12.

1.4 Orchestrator Ortamı

Her iki proje de UiPath Orchestrator platformunda Tasarim_Projesi klasör yapısı altında barındırılmaktadır. Süreç paketleri, merkezi Tenant Processes Feed alanına yayımlanarak çalışmaya hazır hale getirilmiştir.

2. Uygulama ve Sistem Arayüzleri

2.1 Dosya ve Dizin Yapısı (OS Workspace)

Sürecin tüm operasyonel dosyaları C:\RPA_Workspace\Tasarim_Optimizasyon_CEO_Sureci ana dizini altında yönetilmektedir. Bu ana yapı içerisinde yer alan klasörler ve kullanım amaçları şunlardır:

- **01_Report_Template:** Günlük raporun üretilmesi için kopyalanarak kullanılan örnek **Sablon.xlsx** dosyasını tutar.
- **02_Raw_Images:** Ham görsellerin taranması ve kuyruğa eklenmesi için kaynak olarak kullanılır.
- **03_Optimize_Process:** Python motoru tarafından işlenecek olan dosyaların geçici olarak tutulduğu alandır.
- **04_Gemini_Temp:** Gemini servisi tarafından analiz edilecek geçici dosyaların saklandığı klasördür.
- **05_Final_Process:** SEO uyumlu yeni isimleriyle optimize edilen çıktıların son halinin taşındığı hedef dizindir.
- **06_Exceptions:** Hata alan veya iş kuralına takılan dosyaların yönlendirildiği alandır.

- **07_Reports_Archive:** Günlük Excel takip raporlarının tarih bazlı olarak arşivlendiği klasördür.

2.2 Python Motoru

Performer projesi tarafından UiPath Python aktiviteleri kullanılarak çağrılır ve görsellerin baskı standartlarına uygun biçimde 300 DPI çözünürlüğe dönüştürülmesini sağlar.

2.3 Gemini Yapay Zeka Servisi

Performer projesinde ilgili iş akışına **Content Generation** aktivitesi eklenir ve bu aktivite içerisinde Orchestrator üzerinde tanımlanan **Connections** bağlantısı seçilir. Görsellere uygun SEO isimlerinin üretilmesi ve gün sonu kısa rapor özetinin çıkartılması bu sayede gerçekleştirilir.

2.4 Microsoft Excel

Performer projesinin gün sonu işlem sonuçlarını arka planda otomatik olarak şablon dosya üzerinden güncelleyip takip raporuna dönüştürmesi için kullanılır.

2.5 GSuite E-Posta Servisi

Süreç sonunda hazırlanan Excel raporunun ve kısa Gemini özetinin ilgili kişilere iletilmesi için Orchestrator üzerindeki **Gmail Connections** ve dahili UiPath aktiviteleri kullanılır.

3. Teknik Süreç Akışı (Technical Process Flow)

Süreç, REFramework mimarisiyle iki aşamalı olarak tasarlanmıştır:

3.1 Dispatcher Akışı

Dispatcher projesi veri girişini sağlayan temel modüldür. Olası klasör erişim veya bağlantı hatalarını anında yakalayabilmek amacıyla Main dizilimi en dış katmanda ana bir Try Catch bloğu ile tasarlanmıştır.

Try Bloğu Ana İşlem Akışı

- **Init ve Orchestrator Bağlantıları:** Log Message ile süreç başlangıcı bildirilir. Ardından **Get Asset** aktiviteleri ardışık olarak kullanılarak kaynak klasör yolu için **Disp_02_Raw_Images**, hedef kuyruk adı için **Disp_OrchestratorQueueName** ve kuyruk klasör konumu için **Disp_OrchestratorQueueFolder** varlıkları Orchestrator üzerinden çekilir.
- **Kaynak Klasörü Okuma:** Klasör tarama işleminin başladığı loglanır ve Assign aktivitesi içindeki **Directory.GetFiles** komutu kullanılarak hedef klasördeki dosyalar dizi listesine aktarılır. Bulunan toplam dosya sayısı bilgi amaçlı loglanır.
- **Kuyruğu Doldurma İşlemleri:** **For Each** döngüsü ile listedeki her bir dosya tek tek ele alınır. Kuyruğa ekleme öncesi **Log Message** ile bildirim yapılır ve **Add Queue Item** aktivitesi kullanılarak dosyalar **FilePath** anahtarı ile hedef kuyruğa aktarılır. Tüm liste tamamlandığında süreç bitiş logu atılır.

Catch Bloğu Sistem Hatası Yönetimi

- **Süreci Güvenle Durdurma:** Kaynak klasöre ulaşamaması veya varlıkların okunamaması gibi sistem hatalarında akış doğrudan bu bloğa yönlendirilir. Sistem **Log Message** ile Fatal seviyesinde hata bildirimi yapar ve süreci askıda bırakmamak için **Terminate Workflow** aktivitesi kullanılarak akış güvenli bir şekilde sonlandırılır.

3.2 Performer Akışı

Performer projesi, Dispatcher tarafından yüklenen biletleri işleyen ve REFramework standart adımlarını kullanan ana modüldür.

3.2.1 Init Hazırlık Aşaması (Initialization Phase)

3.2.1.1 Ayarların ve Varlıkların Okunması (InitAllSettings)

- **Konum:** Data\Config.xlsx dosyası üzerinden yürütülür ve InitAllSettings akışı ile yüklenir.

- **Teknik İşlem:** Sürecin temel çalışma dizinleri, kuyruk adları ve Python motoruna ait dosya yolları bu yapılandırma tablosundan çekilir. REFramework şablonu bu verileri arka planda otomatik olarak sözlük yapısına aktardığı için kod seviyesinde ek bir aktivite eklenmez.

3.2.1.2 Açık Uygulamaların Kapatılması (KillAllProcesses)

- **Konum:** Framework\KillAllProcesses.xaml iş akışı tetiklenir.
- **Argüman Bağlantısı:** Projenin ana yapılandırma sözlüğü alt akışa in_Config argümanı olarak aktarılır.
- **Uygulama Kontrolü:** Akış başında yer alan If şartı ile yapılandırma dosyasındaki kapatılacak uygulamalar satırının dolu olup olmadığı denetlenir.
- **Liste Ayırıştırma:** Tanımlı metin Regex.Split fonksiyonu ile virgüllerden ayrılarak bir dizi listesine dönüştürülür ve For Each döngüsü başlatılır.
- **Süreç Sonlandırma:** Döngüdeki her bir program Kill Process aktivitesi ile kapatılır. Kapalı olan uygulamalarda robotun durmaması adına ContinueOnError seçeneği aktif tutulur.
- **Atlanma Durumu:** Tabloda temizlenecek bir uygulama bulunmaması halinde sistem Warn seviyesinde log atarak süreci aksatmadan sonraki adıma geçer.

3.2.1.3 Klasör ve Sistem Doğrulaması (InitAllApplications)

- **Konum ve Akış Yapısı:** Framework\InitAllApplications.xaml iş akışı içerisindeki Adım 1.3 - Çalışma Klasörlerinin Kontrolü isimli Sequence bloğunda yürütülür. Sürecin en üstünde sistem hazırlıklarının başladığı **Log Message** ile bildirilir. Bu alt modül tasarım bütünlüğünü korumak adına içeride üç bağımsız **Sequence** bloğundan oluşur.
- **Argüman Bağlantısı:** Ana akışta yer alan **Config** sözlüğü, çağrılan alt akışa in_Config argümanı üzerinden aktarılır.
- **Çalışma Klasörlerinin Kontrolü:** İlgili **Sequence** dizini içerisinde ilk olarak sürecin raporlama yapabilmesi için zorunlu olan **01_Report_Template** şablon klasörünün varlığı bir **If** aktivitesi ile denetlenir. Yapılandırma sözlüğünden çekilen bu klasör sistemde bulunamazsa **Log Message** ile Fatal seviyesinde uyarı verilir ve **Throw** aktivitesi kullanılarak süreç güvenli bir şekilde tamamen durdurulur. Klasör mevcutsa başarılı olduğu loglanıp işleme devam edilir.
- **Klasörlerin Otomatik Oluşturulması:** Aynı **Sequence** bloğunun devamında yapılandırma tablosundan gelen **Calisma_Klasor_Listesi** değeri **Regex** komutuyla parçalanarak bir diziye dönüştürülür ve **For Each** döngüsü başlatılır. Döngü sırasındaki her bir klasörün diskte var olup olmadığı **If** şartı ile taranır. Eksik klasör tespit edilirse sistem Warn seviyesinde log atarak **Create Directory** aktivitesi ile ilgili dizini otomatik olarak oluşturur. Klasör zaten mevcutsa sadece Trace seviyesinde işlem izi bırakılarak atlanır.

3.2.1.4 Günlük Excel Takip Tablosunun Hazırlanması

- **Konum ve Değişkenler:** Framework\InitAllApplications.xaml iş akışı içerisindeki Adım 1.4 Günlük Excel Raporunun Hazırlanması isimli **Sequence** bloğunda yürütülür. İşlem sırasında dinamik tarih ve dosya adreslerini hafızada tutmak üzere **str_TodayDate**, **str_TemplatePath** ve **str_DailyReportPath** değişkenleri tanımlanır.
- **Yol ve Tarih Atamaları:** İşlem başlangıcında atılan **Log Message** sonrasında **Multiple Assign** aktivitesi kullanılarak operasyonel yollar oluşturulur. Sistemden güncel tarih alınarak **str_TodayDate** değişkenine aktarılır. Yapılandırma sözlüğünden şablon dizini çekilerek tam yol **str_TemplatePath** değişkenine, günün tarihiyle arşivde oluşturulacak nihai dosyanın adresi ise **str_DailyReportPath** değişkenine atanır.
- **Şablon Dosyası Doğrulaması:** Ana şablon dosyasının belirtilen sistem yolundaki varlığı bir **If** şartı ile kontrol edilir. Kaynak dosya bulunamazsa Fatal seviyesinde uyarı verilir ve **Throw** aktivitesi kullanılarak süreç başlatılmadan güvenli durdurulur. Şablon dosyası mevcutsa robot doğrusal akışına devam eder.
- **Günlük Raporun Otomatik Kopyalanması:** İkinci bir **If** aktivitesi ile **str_DailyReportPath** konumunda o güne ait Excel raporunun daha önce oluşturulup oluşturulmadığı denetlenir. Rapor arşivde yoksa **Copy File** aktivitesi ile orijinal şablon yeni ismiyle hedef dizine kopyalanır ve kopyalamanın başarılı olduğuna dair sistem logu atılır. Rapor zaten diskte mevcutsa kopyalama işlemi atlanarak sadece Trace seviyesinde işlem izi bırakılır.

3.2.1.5 Python Motoru Varlık Kontrolü

- **Konum ve Değişken Yönetimi:** Framework\InitAllApplications.xaml iş akışı içerisindeki Adım 1.5 Python Motoru Varlık Kontrolü isimli **Sequence** bloğunda yürütülür. Bu adımda ekstra bir değişken tanımlanmaz, yapılandırma sözlüğündeki **Python_Exe_Path** yolu doğrudan argüman olarak kullanılır.
- **Altyapı Doğrulaması:** İşlem başlangıcında atılan bilgilendirme logunun ardından, sürecin gereksinimlerinden olan Python motorunun sistemde kurulu olup olmadığı bir **If** şartı ile denetlenir.
- **Hata Yönetimi ve Süreç Kesintisi:** Belirtilen yolda Python altyapısı bulunamazsa sistem Fatal seviyesinde hata bildirimi yapar ve **Throw** aktivitesi tetiklenerek robot hatalı işlem yapmadan süreci en başından güvenli keser.
- **Başarı Durumunda:** Python motoru sistemde mevcutsa **If** aktivitesinin olumsuz durumu boş bırakılarak robotun doğrusal akışına devam etmesi sağlanır. Kontrolün başarıyla geçildiği Trace seviyesinde loglanarak hazırlık aşaması tamamlanır

3.2.2 Veri Alma Aşaması (Get Transaction Data)

Argüman Bağlantıları ve Veri Akışı Ana akış üzerinden

Framework\GetTransactionData.xaml dosyası çağrılırken süreç verileri ilgili argümanlara şu şekilde bağlanır:

- **Giriş Verileri:** İşlem döngü sırasını takip eden **TransactionNumber** değeri **in_TransactionNumber** argümanına, genel proje ayarlarını taşıyan **Config** sözlüğü ise **in_Config** argümanına aktarılır.
- **Çıkış Verileri:** Kuyruktan çekilen ana bilet **out_TransactionItem** argümanı ile dışarı aktarılacak ana akıştaki **TransactionItem** değişkenine iletilir. Orchestrator loglaması için yapılandırılan **out_TransactionID** argümanı ana süreçteki **TransactionID** değişkenine, **out_TransactionField1** argümanı **TransactionField1** değişkenine ve **out_TransactionField2** argümanı ise **TransactionField2** değişkenine tam olarak eşitlenir.
- **Çift Yönlü Veri:** Bellekte tutulan veri tabloları için **dt_TransactionData** değişkeni **io_dt_TransactionData** argümanı üzerinden alt akışa bağlanır.

3.2.2.1 Kuyruktan Gelen Dosyanın Fiziksel Doğrulanması ve Taşınması

- **Konum ve Değişken Yönetimi:** Framework\GetTransactionData.xaml iş akışı içerisinde yeni bir biletin alındığı **If** aktivitesinin **Then** dalında yürütülür. Kod karmaşasını önlemek adına bu adımdaki tüm işlemler **Seq_VerifyAndMoveFile** isimli **Sequence** bloğu içerisine yerleştirilir. İşlem sırasında kuyruktan gelen dosya yolunu tutmak için **str_FilePath** ve dosyanın fiziksel varlık durumunu tutmak için **bool_FileExists** değişkenleri tanımlanır.
- **Adım 2.2 -Kuyruk Verisinin Okunması:** İlgili **Sequence** dizini içerisinde ilk olarak **Assign** aktivitesi kullanılarak kuyruk ögesindeki hedef dosya yolu çekilir ve **str_FilePath** değişkenine aktarılır. Ardından **File Exists** aktivitesi ile bu dosyanın sistemde gerçekten var olup olmadığı taranır ve sonuç **bool_FileExists** değişkenine aktarılır.
- **Fiziksel Doğrulama ve Taşıma:** Elde edilen varlık durumu bir **If** şartı ile denetlenir. Dosya diskte mevcutsa Trace seviyesinde log atılarak **Move File** aktivitesi ile dosya, yapılandırma tablosundan gelen **03_Optimize_Process** klasörüne taşınır. Taşıma işlemi sonrasında yeni hedef adres **Assign** aktivitesi kullanılarak **str_FilePath** değişkeninin değeri yeni hedef adresle güncellenir.
- **Hata İzolasyonu:** Eğer **If** kontrolünde dosyanın diskte bulunmadığı tespit edilirse sistem süreci tamamen durdurmaz. Bunun yerine **Throw** aktivitesi kullanılarak

Business Exception fırlatılır ve robotun hatalı bileti atlayarak sıradaki işleme güvenle geçmesi sağlanır.

3.2.2.2 Akıllı Kimlik, Raporlama Alanları ve Global Etiketleme (Transaction Field Assignments)

- **Konum ve Akış Başlangıcı:** Framework\GetTransactionData.xaml iş akışı içerisinde **Seq_VerifyAndMoveFile** doğrulama bloğunun hemen altındaki mevcut şablon atama alanında yürütülür. İşlem başlangıcında kimlik ve raporlama alanlarının oluşturulmaya başlandığını bildiren **Log Message** aktivitesi tetiklenir.
- **İşlem Kimliği Ataması:** REFramework şablonunda standart olarak gelen **out_TransactionID** değişkenine ait **Assign** aktivitesi özelleştirilir. Klasik zaman formatı yerine, işlenen dosyanın adı ve güncel tarih birleştirilerek benzersiz bir referans kimliği oluşturulur.
- **Raporlama Alanlarının Hazırlanması:** Orchestrator üzerinde daha detaylı süreç takibi yapabilmek amacıyla, şablondaki boş değerler yerine operasyonel veriler kullanılır. Birinci **Assign** aktivitesi ile **out_TransactionField1** değişkenine dosyanın tam konumu aktarılır. İkinci **Assign** aktivitesi ile **out_TransactionField2** değişkenine işlenen dosyanın fiziksel boyutu hesaplanıp metne çevrilerek atanır.
- **Global Log Etiketleme:** Atama işlemlerinin hemen ardından akışa yeni bir **Add Log Fields** aktivitesi eklenir. Bu aktivite içerisinde DosyaAdi anahtarı oluşturularak dosyanın ismi sisteme tanıtılır. Bu yapı sayesinde robotun atacağı sonraki tüm logların arka planına bu bilgi otomatik olarak eklenir.
- **Kapanış ve İşlem Fazına Geçiş:** Hazırlık aşamasının hatasız tamamlandığını ve robotun yeni etiketleriyle işleme hazır olduğunu bildiren son bir **Log Message** atılarak biletin alınma ve kimliklendirme süreci tamamlanır.

3.2.3 İşlem Aşaması (Process)

Konum ve Değişkenler Sürecin ana kalbini oluşturan bu aşama Framework\Process.xaml dosyası içerisinde yürütülür ve mimari yapı olarak bir **Flowchart** diyagramı üzerine inşa edilmiştir. Akış boyunca ara verileri tutmak ve veri transferlerini sağlamak amacıyla lokal kapsamda dört adet metin değişkeni tanımlanmıştır. Bunlar sırasıyla dosya uzantısını tutan **str_Extension**, ham dosya yolunu barındıran **str_FilePath**, yapay zeka tarafından üretilecek yeni ismi taşıyan **str_SeoName** ve taşıma işlemleri için geçici dosya adresini belirten **str_GeminiFilePath** değişkenleridir.

Argüman Bağlantıları ve Veri Akışı Ana akış üzerinden Framework\Process.xaml dosyası çağrılırken süreç verileri ilgili argümanlara şu şekilde bağlanır:

- **Giriş Verileri:** Kuyruktan çekilen işlem biletini taşıyan **TransactionItem** değişkeni **in_TransactionItem** argümanına, projenin genel yapılandırma ayarlarını barındıran **Config** sözlüğü ise **in_Config** argümanına aktarılır.

- **Çıkış Verileri:** İşlem fazı sonucunda üretilecek olan SEO uyumlu yeni dosya adı, **out_SeoName** argümanı üzerinden dışarı aktarılaraq ana akıştaki **str_SeoName** değişkenine eşitlenir.

Mimari Tasarım ve Akış Şeması

Process.xaml ana dosyası, sürecin yukarıdan aşağıya görsel bir harita sunması ve karar noktalarının net görülmesi için standart bir ardışık dizi yerine **Flowchart** mimarisinde tasarlanır. Ana akışın sade ve temiz kalması için görsel işleme ve isimlendirme gibi adımlar ayrı alt kod dosyalarında kodlanır. Bu alt dosyalar ana şema içerisine **Invoke Workflow File** aktivitesi kullanılarak bağlanır. Yalnızca format kontrolü ve son taşıma/adlandırma işlemi doğrudan ana **Flowchart** üzerinde yürütülür.

Akış Şeması Düğüm Yerleşimi:

- **Başlangıç Noktası:** Süreç **Start** düğümü ile tetiklenir.
- **Birinci Adım:** Başlangıç düğümünden çıkan ok **Multiple Assign** kutusuna bağlanarak bilek içindeki dosya yolu ve uzantı bilgisi değişkenlere aktarılır.
- **İkinci Adım:** Bir önceki kutudan çıkan ok **Flow Decision** karar kutusuna bağlanarak uzantının png formatında olup olmadığı denetlenir.
- **Olumsuz Senaryo:** Karar kutusunun olumsuz çıkış kolu bir **Throw** aktivitesine bağlanarak desteklenmeyen dosya formatı için iş kuralı hatası fırlatılır.
- **Olumlu Senaryo:** Karar kutusunun olumlu çıkış kolu sırasıyla görsel kalitesini artıran **Python_Calistir** iş akışına, yeni isim üreten **Gemini_Baglan** iş akışına ve son olarak dosyayı yeni ismiyle hedef klasöre taşıyan **Seq - Yeniden Adlandır ve Taşı (05_Final_Process)** dizinine ardışık olarak bağlanır.

3.2.3.1 Format Kontrolü (Business Exception Check)

- **Konum ve Değişkenler:** Framework\Process.xaml iş akışı içerisinde ana şema düzeyinde yürütülür. Şema düzeyinde kuyruktan gelen dosya yolunu tutmak üzere **str_FilePath** ve dosya uzantısını tutmak üzere **str_Extension** değişkenleri oluşturulur.
- **Veri Ataması:** Başlangıç düğümüne bağlanan ilk **Multiple Assign** aktivitesi kullanılarak kuyruktaki öğeden dosya yolu çekilip **str_FilePath** değişkenine aktarılır. Aynı aktivite içerisinde dosyanın uzantısı sistem fonksiyonlarıyla küçük harflere dönüştürülerek **str_Extension** değişkenine eşitlenir.
- **Format Doğrulaması:** Elde edilen uzantı **Flow Decision** aktivitesine aktararak verinin png uzantısına sahip olup olmadığı şartı denetlenir.
- **Hata Yönetimi:** Şartın sağlanmaması halinde akış yönü **Throw** aktivitesine iletilir ve desteklenmeyen dosya formatı hatası fırlatılarak robotun bu öğeyi elenmiş bilekler arasına alması sağlanır. Şartın sağlanması halinde ise sistem kesintiye uğramadan bir sonraki işleme yönlendirilir.

3.2.3.2 Görsel Kalitesinin Artırılması (Python_Calistir)

Konum ve Değişkenler: Framework\Process.xaml içerisinde **Flow Decision** karar kutusunun olumlu çıkış koluna bağlanan **Invoke Workflow File** aktivitesi ile yürütülür. Bu alt akış çağrıldığında arka planda çalışacak olan Python motoru üç temel görevi yerine getirir. Orijinal dosya giriş klasöründen okunarak baskı standartlarına uygun olarak çözünürlüğü yükseltilir. İşlenen dosya orijinal ismi korunarak geçici klasöre kaydedilir. Sistem temizliğini sağlamak adına işi biten ham dosya sistemden otomatik olarak silinir. İşlem sırasında Python kodunu hafızada tutmak ve fonksiyonları çalıştırmak için sadece **py_Script** değişkeni kullanılır.

Argüman Bağlantıları: Ana akış üzerinden Framework\Python_Calistir.xaml dosyası çağrılırken süreç verileri ilgili argümanlara şu şekilde bağlanır:

- **Giriş Verileri:** İşlenecek ana dosyanın yolunu taşıyan **str_FilePath** değişkeni **in_FilePath** argümanına, yapılandırma ayarlarını barındıran **in_Config** sözlüğü ise yine aynı isimdeki **in_Config** argümanına aktarılır.
- **Çıkış Verileri:** Python optimizasyonu sonucunda üretilecek olan dosyanın geçici klasördeki yeni adresi, **out_NewFilePath** argümanı üzerinden dışarı aktarılacak ana akıştaki **str_GeminiFilePath** değişkenine tam olarak eşitlenir.

İş Akışı Aktivite Detayları

- **Giriş Yolu Ataması:** İşlem başlangıcında dosya okuma rotasını belirlemek üzere ilk **Assign** aktivitesi kullanılır. Bu aktivite ile yapılandırma sözlüğündeki **03_Optimize_Process** anahtarından okunacak giriş dizini çekilir, işlenecek ham dosyanın tam adresi oluşturulur ve **in_OptimizeFilePath** argümanına aktarılır.
- **Çıkış Yolu Ataması:** İkinci **Assign** aktivitesi ile yapılandırma sözlüğündeki **04_Gemini_Temp** anahtarından çıktı dizini çekilir. Python motoru tarafından çözünürlüğü yükseltilmiş yeni görselin kaydedileceği nihai hedef yol, orijinal dosya ismi korunarak oluşturulur ve **out_NewFilePath** argümanına eşitlenir.
- **İşlem Başlangıç Bildirimi:** Yol atamalarının ardından **Log Message** aktivitesi kullanılarak Trace seviyesinde motorun ham dosyayı işlemeye başladığı sisteme bildirilir.
- **Python Çalışma Ortamı:** Tüm görsel işleme operasyonu **Python Scope** alanı içerisinde yürütülür. Bu ortamın dışa bağımlı olmadan güvenle çalışabilmesi için gerekli olan kurulum ve kütüphane yolları, yapılandırma tablosundaki **Python_Folder_Path** ile **Python_Library_Path** anahtarlarından dinamik olarak çekilir.
- **Kodun Belleğe Alınması:** Kapsam içerisine yerleştirilen **Load Python Script** aktivitesi ile yapılandırma dosyasındaki **tasarim_optimizasyon** anahtarında belirtilen

diskteki Python kod dosyası okunarak **py_Script** değişkeni üzerinden doğrudan robotun belleğine alınır.

- **Fonksiyonun Tetiklenmesi:** Hemen sonrasında **Invoke Python Method** aktivitesi ile yapılandırma sözlüğündeki **Python_Method_Name** anahtarına karşılık gelen fonksiyon çağrılır. Bu aşamada giriş ve çıkış yolları dizi formatında doğrudan çalışan koda iletilir.
- **Kapanış ve Temizlik Bildirimi:** Optimizasyon tamamlandığında **Python Scope** kapsamından çıkılarak **Python_Calistir** ana **Sequence** bloğunun en sonuna eklenen Info seviyesindeki **Log Message** aktivitesi ile dosyanın başarıyla istenen çözünürlük standardına getirildiği, geride kalan orijinal ham kopyanın sistemden temizlendiği ve verinin **Gemini_Baglan.xaml** iş akışına hazır olduğu loglanır.

3.2.3.3 Gemini ile SEO İsimlendirme (Gemini_Baglan)

Konum ve Değişkenler: Framework\Process.xaml içerisinde bir önceki adım olan **Python_Calistir** alt akışından çıkan ok doğrudan bu **Invoke Workflow File** aktivitesine bağlanır. Bu adımda robot, hedef resmi ve yapılandırma dosyasındaki komut metnini yapay zekaya ileterek arama motoru optimizasyonuna uygun bir başlık üretilmesini sağlar. İşlem sırasında motordan dönen ham uzun metni bellekte geçici olarak tutmak için lokal düzeyde **str_GeminiResponse** değişkeni tanımlanır.

Argüman Bağlantıları ve Veri Akışı Ana akış üzerinden Framework\Gemini_Baglan.xaml dosyası çağrılırken süreç verileri ilgili argümanlara şu şekilde bağlanır:

- **Giriş Verileri:** **Python_Calistir** iş akışında hazırlanan geçici dosyanın yolunu tutan **str_GeminiFilePath** değişkeni **in_TempFilePath** argümanına, genel yapılandırma verilerini taşıyan **in_Config** sözlüğü ise yine aynı addaki **in_Config** argümanına aktarılır.
- **Çıkış Verileri:** Yapay zeka analizi sonucunda oluşturulan SEO uyumlu yeni dosya adı, **out_SeoName** argümanı üzerinden dışarı aktarılarak ana akıştaki **str_SeoName** değişkenine tam olarak eşitlenir.

İş Akışı Mantığı ve Aktivite Detayları

- **Başlangıç Bildirimi:** Akış başlangıcında **Log Message** aktivitesi ile Trace seviyesinde log atılarak incelenen dosya adı ve görsel analiz sürecinin başladığı sisteme bildirilir. Böylece olası bağlantı veya zaman aşımı hatalarında takılma noktası anında tespit edilir.
- **Yapay Zeka Görsel Analizi:** Görselin okunması ve yeni ismin üretilmesi işlemi **Image Analysis** aktivitesi ile gerçekleştirilir. Yapay zekanın inceleyeceği resmin adresi olarak **in_TempFilePath** argümanı verilir. Sistemin ne yapması gerektiğini belirten komut metni ise yapılandırma tablosundaki **Perf_Gemini_Prompt**

anahtarından dinamik olarak çekilir. Motor analizini tamamladığında dönen ham cevap metni **str_GeminiResponse** değişkenine aktarılır.

- **Regex ile Format Yakalama:** Yapay zekadan dönen ham metin doğrudan kullanılamayacağı için **Assign** aktivitesi ile bir filtreleme işlemi yapılır. Atama sırasında sistemin **Regex** arama fonksiyonları kullanılarak ham metin içindeki sadece küçük harf, rakam ve tirelerden oluşan, sonunda geçerli bir görsel uzantısı barındıran kesin dosya adı cımbızlanır. Çekilen bu temizlenmiş değer doğrudan **out_SeoName** argümanına eşitlenerek klasör isimlendirmesinde hata yaratmayacak net bir formata sokulur.
- **Hata Yönetimi ve Bildirim:** İsimlendirme sürecinin güvenliğini sağlamak amacıyla akışın sonuna bir **If** karar mekanizması eklenir. Şart bölümünde düzenli ifade işleminin başarısız olup olmadığı ve **out_SeoName** argümanının boş değer döndürüp döndürmediği denetlenir. İsim üretilmemişse sistemin donmasını engellemek için akış **Throw** aktivitesine yönlendirilir ve yapay zeka motorunun ham cevabıyla birlikte iş kuralı hatası fırlatılarak robotun sıradaki dosyaya geçmesi sağlanır. İsim başarıyla üretilmişse Info seviyesindeki **Log Message** aktivitesi ile doğrudan bağlantı kurulan analiz sürecinin başarıyla tamamlandığı ve temizlenen dosya adının sisteme kazandırıldığı onaylanarak alt akış sonlandırılır.
- **Kapanış:** Alt akıştan çıkılarak elde edilen yeni dosya adı **out_SeoName** argümanı üzerinden doğrudan Framework\Process.xaml ana akışındaki **str_SeoName** değişkenine teslim edilir.

3.2.3.4 Yeniden Adlandırma ve Taşıma (Seq - Yeniden Adlandır ve Taşı)

Konum ve Değişkenler

Framework\Process.xaml içerisinde **Gemini_Baglan.xaml** adımından çıkan ok, doğrudan ana şema üzerine yerleştirilmiş **Seq - Yeniden Adlandır ve Taşı** adlı **Sequence** bloğuna bağlanır; blok içerisinde dosya yolları ve adlandırma için **str_GeminiFilePath** ile **str_SeoName** değişkenleri kullanılır.

İş Akışı Mantığı ve Aktivite Detayları

- **Başlangıç Bildirimi:** Kutu içerisindeki ilk adımda **Log Message** aktivitesi kullanılarak Trace seviyesinde log atılır ve dosyanın yeni ismiyle **05_Final_Process** klasörüne taşınmaya başlandığı sisteme bildirilir.
- **Taşıma ve İsimlendirme:** Operasyonun merkezinde yer alan **Move File** aktivitesi mevcut dosyayı **04_Gemini_Temp** klasöründen alır. Hedef yol belirlenirken sistem yol birleştirme fonksiyonları yardımıyla yapılandırma sözlüğünden çekilen **05_Final_Process** klasör adresi ile yeni dosya ismi bir araya getirilir. Bu sayede dosya tek bir hamlede hem belirtilen yeni hedefe aktarılır hem de arama motoru optimizasyonuna uygun şekilde yeniden adlandırılır.

- **Başarı Bildirimi:** Fiziksel taşıma işlemi bittiğinde Info seviyesinde yeni bir **Log Message** aktivitesi çalıştırılarak dosyanın **05_Final_Process** klasörüne başarıyla ulaştığı onaylanır.
- **Çıkış Ataması ve Kapanış:** Akışın en sonuna eklenen **Assign** aktivitesi kullanılarak, üretilen yeni dosya adını tutan yerel **str_SeoName** değişkeninin değeri doğrudan **out_SeoName** çıkış argümanına aktarılır ve akış sonlandırılır.

3.2.4 Hatalı Dosyanın İstisna Klasörüne Taşınması (SetTransactionStatus)

Konum ve Kapsam Bu aşamadaki tüm operasyonlar Framework\SetTransactionStatus.xaml dosyası içerisinde yer alan ilgili durum bloklarında yürütülür: Hatalı dosyaların taşınması **Business Exception** ve **System Exception** alanlarında gerçekleştirilirken, başarılı işlemlerin raporlanması ve kuyruğa veri yazılması **Success** bloğu altında tamamlanır

Argüman Bağlantıları ve Veri Akışı

Ana akış üzerinden Framework\SetTransactionStatus.xaml dosyası çağrılırken süreç verileri ve durum bildirimleri ilgili argümanlara şu şekilde bağlanır:

- **Giriş Verileri:** Yapay zeka tarafından üretilen yeni dosya adı **str_SeoName** değişkeninden **in_SeoName** özel argümanına aktarılır. İş kuralı hatası detayları **BusinessException** değişkeninden **in_BusinessException** argümanına iletilir. Kuyruk ögesine ait birinci işlem alanı **TransactionField1** değişkeninden **in_TransactionField1** argümanına, ikinci işlem alanı **TransactionField2** değişkeninden **in_TransactionField2** argümanına, log kimlik bilgisi ise **TransactionID** değişkeninden **in_TransactionID** argümanına bağlanır. Projenin yapılandırma ayarları **Config** sözlüğünden **in_Config** argümanına, üzerinde çalışılan işlem bileti ise **TransactionItem** değişkeninden **in_TransactionItem** argümanına teslim edilir. Blok iş kuralı hatası bağlamında tetiklendiği için sistem hatası **Nothing** değeri ile **in_SystemException** argümanına gönderilir.
- **Çift Yönlü Veriler:** Sistemin döngüsel takibini ve ardışık hata limitlerini yöneten sayısal değerlerden **RetryNumber** değişkeni **io_RetryNumber** argümanına, **TransactionNumber** değişkeni **io_TransactionNumber** argümanına ve **ConsecutiveSystemExceptions** değişkeni **io_ConsecutiveSystemExceptions** argümanına tam eşleşme ile bağlanır. Bu sayede statü güncellemesi sonrasında artırılan ya da sıfırlanan yeni değerlerin ana şemaya eksiksiz dönmesi sağlanır.

İş Akışı ve Genel Kapsam

İş kurallarına uymayan veya sistem hatası alan dosyaların kaynak klasörde kalıp birikmesini önlemek amacıyla otomatik taşıma işlemi yapılır. Bu işlem kurumsal REFramework mimarisini bozmamak adına Framework\SetTransactionStatus.xaml dosyası içerisindeki ilgili hata bloklarının en üstüne eklenen özel bir **Sequence** aracılığıyla gerçekleştirilir.

3.2.4.1 İş Kuralı Hatası Kurulumu (Business Exception)

- **Konum :** Framework\SetTransactionStatus.xaml dosyası içerisindeki **Business Exception** kutusuna girilir. En üstte bulunan ve kuyruk ögesi kontrolünü yapan **If TransactionItem is a QueueItem - Business Exception** aktivitesinin hemen üzerine yeni bir **Sequence** sürüklenerek adı tam olarak **Hatalı Dosyayı Taşıma** şeklinde değiştirilir.
- **Başlangıç Bildirimi:** Eklenen bu yeni kutunun içerisindeki ilk adımda **Log Message** aktivitesi kullanılarak Warn seviyesinde log atılır. İşlem yapılan dosyanın dinamik yolu belirtilerek hedefin **06_Exceptions** klasörü olduğu sisteme bildirilir.
- **Taşıma Operasyonu:** Bildirimin hemen altına eklenen **Move File** aktivitesi ile hatalı dosya dinamik kaynak yolundan alınır. Hedef yol olarak yapılandırma sözlüğündeki **06_Exceptions** adresi gösterilir.

3.2.4.2 Sistem Hatası Kurulumu (System Exception)

- **Konum :** Framework\SetTransactionStatus.xaml dosyası içerisindeki **System Exception** kutusuna girilir.
- **Kopyalama ve Entegrasyon:** İş kuralı hatası bölümünde tüm parametreleri eksiksiz hazırlanan **Hatalı Dosyayı Taşıma** isimli **Sequence** kutusu tamamen kopyalanır. Kopyalanan bu blok sistem hatası dizisinin en tepesinde yer alan ve ardışık hataları bildiren **Log Message** aktivitesinin hemen üzerine yapıştırılarak taşıma mekanizması iki hata senaryosu için de standart hale getirilir.

3.2.4.3 Yeni Dosya Adını Kuyruğa Ekleme (Success)

İş Akışı Mantığı ve Genel Kapsam

Raporlama aşamasında nihai Excel belgesine yeni dosya isminin eksiksiz yazılabilmesi amacıyla, işlemi başarıyla tamamlanan dosyanın yeni adı doğrudan Orchestrator kuyruk ögesinin çıktı verilerine kaydedilir. Bu operasyon, ana akışın başarılı sonuçlanması durumunda devreye giren Framework\SetTransactionStatus.xaml dosyası içerisindeki ilgili blokta gerçekleştirilir.

Uygulama Adımları ve Aktivite Detayları

- **Konum:** İşlem, Framework\SetTransactionStatus.xaml dosyası içerisindeki **Success** bloğunda yer alan **Try Catch Set Transaction Status - Success** dizisinin **Try** bölümünde bulunan **Set transaction status - Successful** aktivitesi üzerinde gerçekleştirilir.

- **Özellik Yapılandırması ve Veri Ataması:** İlgili aktivitenin sağ taraftaki özellikler panelinden **Output** koleksiyonu penceresi açılır. Kuyruk üzerinde tutulacak verinin anahtar kelimesi olarak metin tipinde **SEO_Name** ifadesi girilir ve bu anahtarın karşılık değeri olarak yapay zeka aşamasından elde edilerek dış argümanlar üzerinden buraya kadar taşınan **in_SeoName** değişkeni atanarak yapılandırma tek adımda tamamlanır.

3.2.5 Sürecin Sonlandırılması ve Kapanış (End Process)

- **Konum:** İlgili alt akış, Framework\EndProcess.xaml ana sayfası içerisindeki **Try to close all applications** dizisinin hemen üzerine eklenen **Invoke Workflow File** aktivitesi ile **Modules\Rapor_Ve_Mail_Yonetimi.xaml** dosyası çağrılarak tetiklenir.
- **Argüman Bağlantıları:** Rapor_Ve_Mail_Yonetimi.xaml dosyasında süreç genelindeki klasör, kuyruk ve e-posta yapılandırma ayarlarını aktarmak amacıyla **Config** sözlüğü doğrudan **in_Config** argümanına bağlanır.
- **Değişkenler:** Süreç boyunca operasyonu yönetmek ve veri transferini sağlamak amacıyla şu argüman ve değişkenler kullanılır:
 - o **kuyrukItemlari:** Kuyruktan çekilen tüm işlem biletlerini veri işleme aşaması için hafızada toplu olarak tutan değişkendir.
 - o **dt_Rapor:** İşlem verilerinin fiziksel Excel dosyasına aktarılmadan önce işlendiği sanal veri tablosu değişkenidir.
 - o **str_RaporYolu:** Excel dosyasının dinamik tam yolunu tek bir merkezde tutarak kod tekrarını önleyen metin değişkenidir.
 - o **str_TabloMetni:** Yapay zekaya gönderilmek üzere veri tablosunun düz metin haline getirilmiş kopyasını barındıran değişkendir.
 - o **str_OzetMetin:** Yapay zekadan dönecek olan nihai özet analiz metnini tutan değişkendir.

İş Akışı Mantığı ve Ön Hazırlık Adımları Doğrusal bir çalışma prensibine sahip olan bu modül, en dışta **Seq_Rapor_Ve_Mail_Yonetimi** adlı tek bir ana dizi üzerinden tasarlanmıştır. Kodun okunabilirliğini artırmak ve hata ayıklama süreçlerini kolaylaştırmak amacıyla bu ana dizinin içerisine sırasıyla **Sequence - Veri Çekme ve Tablo Oluşturma**, **Sequence - Excel Raporlama**, **Sequence - Gemini Genel Özet Analizi** ve **Sequence - Mail Bildirimi ve Kapanış** olmak üzere dört ayrı alt dizi yerleştirilmiştir.

- **Rapor Yolunun Belirlenmesi:** Sürecin en başında, ana dizi olan **Seq_Rapor_Ve_Mail_Yonetimi** içerisinde yer alan **Assign** aktivitesi kullanılarak dinamik bir dosya rotası oluşturulur. Yapılandırma sözlüğünden çekilen

07_Reports_Archive klasör adresi ile günün tarihini taşıyan Excel dosya adı sistem yol birleştirme fonksiyonları yardımıyla birleştirilerek **str_RaporYolu** değişkenine atanır.

- **Başlangıç Bildirimi:** Atama işleminin hemen ardından aynı ana dizi içerisinde Info seviyesinde bir **Log Message** aktivitesi çalıştırılarak gün sonu raporlama ve e-posta iletim sürecinin başlatıldığı sisteme bildirilir.

3.2.5.1 Veri Çekme ve Tablo Oluşturma (Sequence)

Bu aşamada Orchestrator üzerindeki işlem biletleri toplu olarak çekilir ve raporlanmak üzere sanal bir veri tablosu şablonuna işlenir.

- **Kuyruk Verilerinin Alınması:** Süreç **Get Queue Items** aktivitesi ile başlar. Yapılandırma sözlüğünde yer alan kuyruk adı referans gösterilerek tüm biletler **kuyrukItemlari** koleksiyon değişkenine aktarılır. İşlemin hemen ardından Trace seviyesinde bir **Log Message** aktivitesi çalıştırılarak verilerin alındığı ve şablon oluşturma safhasına geçildiği sisteme bildirilir.
- **Sanal Tablo İnşası:** Bellek üzerinde rapor iskeletini kurmak amacıyla **Build Data Table** aktivitesi kullanılır. Oluşturulan bu tablo **dt_Rapor** değişkenine atanır ve içerisine metin formatında dört temel sütun eklenir: Orijinal Dosya Adı, İşlem Statüsü, Yeni SEO Adı ve Hata Detayı.
- **Veri Filtreleme ve Döngü İşlemleri:** Çekilen biletler **For Each** aktivitesi yardımıyla sırayla işleme sokulur ve her bir döngü adımı **currentItem** olarak adlandırılır. Döngü içerisine yerleştirilen **If** aktivitesi kullanılarak biletin başlama zamanı kontrol edilir ve sadece bugünün tarihine ait işlemler filtrelenir.
- **Sanal Tabloya Kayıt:** Tarih koşulunu sağlayan güncel biletler **Add Data Row** aktivitesi ile satır satır sanal tabloya eklenir. Bu veri dizilimi sırasında orijinal dosyanın adı, işlemin başarılı veya hatalı olma durumu, yapay zeka tarafından üretilen yeni dosya ismi ve eğer işlem hatalıysa ilgili hatanın detayı ilgili sütunlara dinamik olarak yazdırılır.

3.2.5.2 Excel Raporlama (Sequence)

Bu kısa aşama, bellek üzerinde hazırlanan ve filtrelenen sanal veri tablosunun fiziksel bir Excel dosyasına dönüştürülmesi işlemini kapsar.

- **Fiziksel Yazdırma Bildirimi:** Veri aktarımından hemen önce Trace seviyesinde bir **Log Message** aktivitesi tetiklenir. Bu bildirim sayesinde sanal tablonun başarıyla

hafızaya alındığı ve dinamik yol barındıran **str_RaporYolu** değişkenindeki hedefe yazdırılacağı log kayıtlarına işlenir.

- **Dosyaya Aktarım:** Fiziki dosya oluşturma ve verileri kaydetme işlemi **Append Range Workbook** aktivitesi ile gerçekleştirilir. Hedef dizin olarak **str_RaporYolu** değişkeni belirtilir, bellekteki **dt_Rapor** tablosu kaynak olarak gösterilir ve tüm veriler hedeflenen Excel belgesinin birinci sayfasına başarıyla yazdırılarak raporlama kısmı tamamlanır.

3.2.5.3 Gemini Genel Özet Analizi (Sequence)

Bu aşamada oluşturulan Excel tablosundaki veriler düz metne çevrilerek Gemini'ye iletilir bir gün sonu özeti oluşturulur.

- **Metin Dönüşümü:** Çoklu veri barındıran rapor tablosunu yapay zekaya uygun formatta gönderebilmek için **Output Data Table** aktivitesi kullanılır. Bellekteki **dt_Rapor** değişkeni bu aktivite sayesinde düz metin formatına çevrilerek **str_TableMetni** değişkenine aktarılır.
- **Darboğaz Öncesi Bildirim:** Modelin yanıt süresine bağlı olası gecikmeleri log üzerinden izleyebilmek amacıyla Trace seviyesinde bir **Log Message** aktivitesi çalıştırılarak tablo verilerinin yapay zekaya gönderildiği sisteme bildirilir.
- **Yapay Zeka Analizi ve Yorumlama:** Özet metnini oluşturmak için **Content Generation** aktivitesi tercih edilir ve model olarak **gemini-2.5-flash** motoru seçilir. Sistem yönlendirmesi için standart yardımcı asistan tanımı yapılırken ana görev kısmında Orchestrator varlıklarından çekilen **Perf_Gemini_Ozet_Prompt** komutu ile alt satıra geçiş karakteri ve **str_TableMetni** değişkeni birleştirilir. Yapay zekadan dönen detaylı analiz metni doğrudan **str_OzetMetin** değişkenine kaydedilir.
- **Başarı Bildirimi:** Analiz tamamlandığında Info seviyesinde yeni bir **Log Message** aktivitesi devreye girer. Elde edilen özet metnin karakter uzunluğu hesaplanarak işlemin başarıyla sonuçlandığı log kayıtlarına işlenir.

3.2.5.4 Mail Bildirimi ve Kapanış (Sequence)

Sürecin bu son adımında hazırlanan fiziksel Excel dosyası ve yapay zeka özeti ilgili hedef kitleye iletilerek raporlama akışı güvenli şekilde sonlandırılır.

- **İletim Öncesi Bildirim:** Operasyon başlamadan hemen önce Trace seviyesinde bir **Log Message** aktivitesi çalıştırılır. Yapılandırma dosyasındaki **Perf_HedefMailAdresi** verisi kullanılarak hedeflenen alıcıya özet metin ve Excel raporunun gönderilmek üzere olduğu sisteme kaydedilir.
- **Elektronik Posta Gönderimi:** GSuite entegrasyonu kullanan **SendEmailConnections** aktivitesi ile iletim işlemi gerçekleştirilir. Yapılandırma dosyasından okunan **HedefMailAdresi** verisi alıcı kısmına yazılır ve standart gün sonu işlem raporu başlığı eklenir. E-posta gövdesine yapay zekanın ürettiği

str_OzetMetin değişkeni yerleştirilirken fiziksel belge rotasını barındıran **str_RaporYolu** değişkeni ise dosya eki olarak aktiviteye tanımlanır.

- **Süreç Kapanış Bildirimi:** Gönderim işlemi başarıyla gerçekleştiğinde Info seviyesinde son bir **Log Message** aktivitesi tetiklenir. E-postanın hedeflenen adrese ulaştığı sisteme bildirilerek raporlama alt akışı tamamen sonlandırılır.

3.2.5.5 Uygulamaların Kapatılması (CloseAllApplications)

Olası arka plan işlemlerini ve Excel pencerelerini sonlandırma görevi KillAllProcesses akışı tarafından doğrudan üstlenildiği ve süreçte açık bırakılan harici bir arayüz bulunmadığı için Framework\CloseAllApplications.xaml dosyasına ilave bir kapatma adımı eklenmemiş, varsayılan şablon yapısı korunmuştur.

4. Veri Modeli ve Konfigürasyon

4.1 Yapılandırma ve Assets Yönetimi

Süreç parametreleri iki farklı yöntemle yönetilmektedir:

- **Dispatcher Varlıkları:** Dispatcher projesinde Config dosyası bulunmaz; süreç için gerekli ortam bilgileri doğrudan Orchestrator varlıkları üzerinden Get Asset aktiviteleriyle çekilir.
- **Performer Yapılandırma Dosyası:** Performer projesi ise Data klasöründeki Config.xlsx dosyasını kullanır; yerel ayarlar ve Orchestrator varlık adları bu dosya üzerinden dinamik olarak okunur.

Settings Parametreleri:

Ayar Adı	Kullanım Amacı
OrchestratorQueueName	Verilerin aktarılacağı ana kuyruğun adını belirtir.
OrchestratorQueueFolder	Kuyruğun bulunduğu Orchestrator klasör yapısını belirtir.

01_Report_Template	Raporlama için kullanılacak kaynak şablon klasörünün yolunu tutar.
02_Raw_Images	Ham görsellerin bulunduğu kaynak klasörün yolunu tutar.
03_Optimize_Process	Optimizasyon işlemleri için kullanılacak çalışma klasörünün yolunu tutar.
04_Gemini_Temp	Yapay zeka süreçleri için kullanılacak geçici klasörün yolunu tutar.
05_Final_Process	İşlemi tamamlanan nihai dosyaların aktarılacağı klasör yolunu tutar.
06_Exceptions	Hatalı işlem gören dosyaların taşınacağı klasör yolunu tutar.
07_Reports_Archive	Gün sonu raporlarının arşivlendiği klasör yolunu tutar.
Kapatılacak_Uygulamalar	Süreç başlangıcında arka planda açık olan ve sıfırlanması gereken programların listesini barındırır.
Calisma_Klasor_Listesi	Robotun başlangıçta sistemde varlığını kontrol edeceği ve yoksa oluşturacağı klasör adlarının listesini tutar.
Python_Exe_Path	Bilgisayara kurulu olan Python motorunun (exe) çalıştırılabilir dosya yolunu belirtir.
Python_Folder_Path	Python altyapısının kurulu olduğu ana dizin yolunu belirtir.

Python_Library_Path	Python kütüphane (.dll) dosyasının bulunduğu yolu belirtir.
tasarim_optimizasyon	Süreçte kullanılacak olan Python script dosyasının adını belirtir.
Python_Method_Name	Python scripti içerisindeki tetiklenecek ana fonksiyonun adını belirtir.

Assets Parametreleri:

Assets Adı	Kullanım Amacı
Disp_02_Raw_Images	Dispatcher robotunun tarayacağı ham görsellerin bulunduğu kaynak klasör yolunu tutar.
Disp_OrchestratorQueueName	Dispatcher tarafından verilerin aktarılacağı hedef Orchestrator kuyruğunun adını barındırır.
Disp_OrchestratorQueueFolder	İlgili kuyruğun Orchestrator üzerinde bulunduğu klasör (Folder) yolunu belirtir.
Perf_Gemini_Prompt	Yapay zekaya gönderilecek görsel analiz ve SEO isimlendirme komutunu tutar.
Perf_Gemini_Ozet_Prompt	Gün sonu işlem tablosunu analiz edip genel bir özet çıkartacak yapay zeka komutunu tutar.

Perf_HedefMailAdresi	Süreç sonunda hazırlanan özetin ve raporun iletileceği hedef e-posta adresini barındırır.
-----------------------------	---

4.2 Kuyruk Mimarisi ve Veri İletişimi

Dispatcher ve Performer projelerinin birbirlerinden izole ancak tam senkronize çalışmasını sağlayan ana köprü yapı Orchestrator kuyruk mimarisidir.

- **Veri Girişi:** Dispatcher projesi, kaynak klasörde taradığı her bir görselin tam dosya yolunu hedef kuyruğa **FilePath** etiketiyle yeni bir işlem bileti olarak ekler.
- **Veri İşleme ve Çıktı:** Performer projesi ilgili bileti kuyruktan alıp süreci başlatır. İşlem hatasız tamamlandığında, yapay zekanın ürettiği yeni dosya adını **SEO_Name** etiketiyle aynı biletin çıktı bölümüne doğrudan kaydeder.
- **Raporlama Bütünlüğü:** Kuyruktaki işlem sonuçları ve yeni dosya isimleri gün sonunda topluca okunarak Excel raporuna yazılır. Bu sayede tüm sürecin takibi ve veri akışı kuyruk üzerinden eksiksiz sağlanır.

5. Hata Yönetimi (Exception Handling)

5.1 System Exception

Sistem kaynaklı teknik aksaklıkları, erişim ve uygulama hatalarını kapsar.

- **Dispatcher Süreci:** Süreç en dış katmanda bir Try-Catch bloğu ile korunur. Hedef klasöre veya Orchestrator varlıklarına erişilememesi durumunda süreç Fatal log atarak Terminate Workflow aktivitesiyle anında ve güvenli durdurulur.
- **Performer Süreci:** Python altyapısının yanıt vermemesi, sistem donması veya klasör oluşturulmaması gibi durumlarda devreye girer. İşlemi yarıda kalan dosya SetTransactionStatus aşamasında otomatik olarak **06_Exceptions** klasörüne taşınır ve süreç bir sonraki biletle çalışmaya devam eder.

5.2 Business Exception

Sistemsal bir çökme olmayan ancak sürecin iş kurallarına uymayan verilerle karşılaşıldığında fırlatılan kontrollü hatalardır.

- **Tetiklendiği Senaryolar:** Kuyruktan gelen dosyanın diskte fiziksel olarak bulunamaması, gelen uzantının baskı formatına uygun olmaması veya Gemini yapay zeka servisinin istenilen kurallarda bir SEO ismi üretememesi, boş dönmesi durumlarında Business Exception fırlatılır.

- **İzolasyon:** İş kuralı hatası alan işlemler tekrar denenmez. Hatalı dosya, ana klasörde birikme yapmaması için için **06_Exceptions** klasörüne taşınır ve Orchestrator kuyruğundaki statüsü Business Exception olarak güncellenerek kapatılır.

6. Güvenlik ve Varlıklar

6.1 Varlık Yönetimi

Süreçte hiçbir bilgi kod içine sabit olarak yazılmaz. Raporların gideceği e-posta adresi, yapay zeka komutları, klasör yolları ve kuyruk adları doğrudan Orchestrator varlıkları üzerinden güvenli şekilde okunur.

6.2 Sistem Bağlantıları

Gemini yapay zeka servisi ve GSuite e-posta işlemleri için kod içinde hiçbir şifre veya anahtar tutulmaz. İlgili UiPath aktiviteleri, doğrudan aktivite paneli üzerinden Orchestrator Connections yapısına bağlanarak yetkilendirmeyi güvenli bir şekilde sağlar.

6.3 Hata ve İstisna Güvenliği

- **Dispatcher Hata Güvenliği:** Süreç başlangıcında kritik bir hata oluşursa akış Try Catch bloğunda yakalanır ve Terminate Workflow aktivitesiyle kontrollü biçimde sonlandırılır.
- **Performer İstisna İzolasyonu:** İş kuralına uymayan veya sistem hatası alan görseller, ana akışı tıkamaması ve veri güvenliğinin korunması için anında **06_Exceptions** klasörüne taşınarak izole edilir. Robot süreci kitlemeden bir sonraki kuyruk kaydını işlemeye devam eder.

